Methodology & Continuous Monitoring Strategy

1. How the Data Was Analyzed

To ensure recommendations were rooted in current, objective data rather than assumptions, I built an automated extraction and triage pipeline:

1. **Extraction:** I wrote a Python script utilizing the `google-play-scraper` and `pandas` libraries to extract the 1,000 most recent reviews for the MoveInSync Android app directly from the Google Play API.
2. **NLP Triage:** The script filtered out positive reviews ($>= 3$ stars) and utilized a keyword-based Natural Language Processing algorithm to categorize the remaining text into predefined technical buckets (`Auth_Issue`, `Engineering_Bug`, `Location_Issue`, `Operations`).
3. **Artifact Generation:** The script automatically calculated an App Health Score and serialized the most critical crashes into a Jira-compatible JSON payload (`mock_jira_tickets.json`).

2. Continuous Monitoring Architecture ("Not a one-time ask")

MoveInSync explicitly noted that tracking ratings is a continuous requirement. Relying on humans to manually scrape and read reviews is inefficient and prone to error.

To ensure the product and engineering teams always have real-time visibility into user sentiment, I propose deploying the Python script I wrote into a **Continuous Monitoring CI/CD Pipeline**.

Proposed Deployment (GitHub Actions)

By committing the Python script to the MoveInSync repository, we can configure a GitHub Actions workflow to run autonomously every week.

Infrastructure-as-Code Configuration (YAML):

```
name: Weekly Play Store Triage
on:
  schedule:
    - cron: '0 0 * * 0' # Runs automatically every Sunday at midnight

jobs:
  run-app-triage:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v3

      - name: Setup Python Environment
```

```
      uses: actions/setup-python@v4
      with:
        python-version: '3.10'

    - name: Install Dependencies
      run: pip install google-play-scraper pandas

    - name: Execute Triage Script
      run: python play_store_tracker.py

    - name: Push Critical Bugs to Jira API
      run: |
        curl -D- -u admin:api_token \
        -X POST \
        --data @mock_jira_tickets.json \
        -H "Content-Type: application/json" \
        https://moveinsync.atlassian.net/rest/api/2/issue/bulk

    - name: Alert Engineering on Slack
      run: |
        curl -X POST -H 'Content-type: application/json' \
        --data '{"text":"🚨 Weekly Play Store Triage Complete. Check
        ${{ secrets.SLACK_WEBHOOK_URL }}
```

Business Value

This approach transitions MoveInSync from *reactive* Play Store monitoring to *proactive* incident resolution. The Engineering team receives structured bug reports directly in their existing sprint backlog, completely removing manual overhead.